

ChartEdge AI

NSE Technical Analysis SaaS — Paper Trading to Full Automation

PRODUCT REQUIREMENTS DOCUMENT + SYSTEM ARCHITECTURE DESIGN

Version 1.0 | Target: Nifty 50 & Nifty Bank | Exchange: NSE India
Data Source: INDstocks API | AI Engine: Claude (Anthropic)

1. Vision & Product Goals

ChartEdge AI is an AI-powered technical analysis SaaS platform designed exclusively for NSE India, targeting Nifty 50 and Nifty Bank instruments. The system ingests real-time and historical OHLCV data from the INDstocks API, computes a multi-indicator technical confluence stack, and passes enriched chart snapshots to an AI reasoning engine (Claude) which generates BUY/SELL/HOLD signals with confidence scores, risk parameters, and trade rationale. The platform starts as a paper trading simulator and is architected to plug into live order execution (INDMoney API) as a Phase 2 upgrade.

1.1 Core Objectives

- Maximum accuracy of intraday signal generation using pure technical chart analysis
- AI-assisted confluence — no single-indicator decisions; always require multi-indicator agreement
- Full paper trading loop with P&L tracking, trade log, and accuracy reporting for minimum 1 month
- Modular, expandable architecture: new indicators, new instruments, and live execution can be added without re-architecting
- Near real-time operation: signal latency under 5 seconds from candle close to signal output

1.2 Inspiration & Differentiation

Platform	What They Do	ChartEdge Differentiator
Menthor Q	AI-powered options flow & scanner	Focus: pure chart TA with AI reasoning transparency
Danelfin	ML stock scoring & ratings	Focus: intraday NSE signals, not hold ratings
Tickeron	Pattern recognition & AI signals	Focus: INDstocks real-time feed + India-specific logic
Trade Ideas	US market AI scanner	Focus: NSE-native, Nifty/BankNifty specialist

2. Scope, Constraints & Assumptions

2.1 In Scope — Phase 1 (Paper Trading)

- Instruments: Nifty 50 Index (^NSEI) and Nifty Bank Index (^NSEBANK) only
- Market hours: 09:30 AM – 03:15 PM IST, Monday–Friday (NSE schedule)
- Timeframes: 1-min, 5-min, 15-min candles as primary; 1-hr and daily as higher-timeframe filters
- Data source: INDstocks API (REST for historical + WebSocket for real-time streaming)
- Signal output: BUY / SELL / HOLD with confidence %, entry zone, SL, Target 1, Target 2
- Paper trade execution: simulated fills at signal candle close price
- Dashboard: live signal feed, open positions, P&L, trade log, accuracy metrics
- AI Engine: Claude API (claude-sonnet) for chart reasoning and signal narrative

2.2 Out of Scope — Phase 1

- Live brokerage execution (Phase 2 via INDMoney API)
- Options / F&O trading (Phase 3)
- Other NSE/BSE stocks beyond Nifty 50 and Bank Nifty
- Mobile app (web-only in Phase 1)

2.3 Key Constraints

Constraint	Detail
Market Window	Signals only generated between 09:30–03:15 IST; system idles outside hours
API Rate Limits	INDstocks API rate limits must be respected; implement exponential backoff
Candle Lag	Signals fire on confirmed candle close only — no look-ahead bias
AI Latency	Claude API call per signal should complete within 3–4 seconds
Capital Simulation	Paper account starts at ₹10,00,000 simulated capital

3. Functional Requirements

3.1 Module 1 — Market Data Ingestion

3.1.1 Historical Data Loader

- On startup, pre-fetch last 90 days of OHLCV data for both instruments across all required timeframes (1m, 5m, 15m, 1hr, 1D)
- Store in local time-series DB (TimescaleDB or InfluxDB)
- Support backfill — detect gaps and re-fetch missing candles automatically
- Normalize timestamps to IST and forward-fill exchange holidays

3.1.2 Real-Time WebSocket Feed

- Connect to INDstocks WebSocket on market open (09:25 AM)
- Subscribe to tick-level data for both instruments

- Aggregate ticks into 1-min candles in memory using OHLCV accumulator
- Emit 'candle_closed' event at each minute boundary to trigger indicator computation
- Auto-reconnect with exponential backoff on disconnect
- Heartbeat check every 30 seconds; log feed health to monitoring

3.2 Module 2 — Technical Indicator Engine

All indicators computed server-side in Python using pandas-ta or TA-Lib. Indicator values are computed fresh on every candle_closed event. Results stored in indicator cache (Redis) keyed by instrument + timeframe + candle_timestamp.

Core Indicator Stack (Minimum Required)

Indicator	Parameters	Purpose	Timeframes
RSI	Period 14	Momentum & overbought/oversold	5m, 15m, 1hr
MACD	12/26/9 EMA	Trend direction & crossovers	5m, 15m
Bollinger Bands	20 SMA, 2 SD	Volatility & mean-reversion zones	5m, 15m
EMA Ribbon	9, 21, 50, 200 EMA	Trend strength & dynamic S/R	15m, 1hr, 1D
VWAP	Session VWAP	Intraday fair value reference	1m, 5m
ATR	Period 14	Volatility for SL/Target sizing	5m, 15m
Supertrend	ATR 10, Factor 3	Trend filter & trailing stop	15m
Stochastic RSI	K=3, D=3, RSI=14	Momentum confirmation	5m, 15m
Volume MA	20-period SMA of volume	Volume spike detection	5m
Pivot Points	Daily CPR, S1/R1/S2/R2	Key support/resistance levels	Daily

Confluence Scoring

- Each indicator produces a directional vote: +1 (bullish), -1 (bearish), 0 (neutral)
- Weighted confluence score computed: $\sum(\text{indicator_vote} \times \text{weight}) / \sum \text{weights}$
- Score > +0.6 triggers BUY consideration; Score < -0.6 triggers SELL consideration
- Score between ±0.6 results in HOLD — no signal fired
- Weights configurable per instrument; default weights stored in config YAML

3.3 Module 3 — AI Signal Engine (Claude Integration)

When confluence score crosses threshold, the system constructs a structured prompt payload and sends it to Claude API for final signal reasoning and narrative generation.

3.3.1 Prompt Payload Structure

- Instrument name, current candle OHLCV (5m and 15m)
- All indicator values with their directional state (e.g., RSI=68 OVERBOUGHT, MACD=BULLISH CROSSOVER)
- Higher timeframe context: 1hr trend direction, daily trend, proximity to key S/R
- Recent 20-candle OHLCV summary (open, high, low, close as compact array)
- Pre-computed confluence score and individual indicator votes
- Market context: time of day, session phase (morning/midday/expiry), day of week

3.3.2 Claude Output Schema (JSON)

- signal: BUY | SELL | HOLD
- confidence: 0–100 integer (percent)
- entry_zone: { low: price, high: price }
- stop_loss: price
- target_1: price
- target_2: price
- risk_reward_ratio: float
- reasoning: string (max 3 sentences — why this signal)
- warnings: array of strings (e.g., 'Near major resistance', 'Low volume')
- invalidation: string (what would invalidate this trade)

3.3.3 AI Engine Rules

- Claude must never override a HOLD if confluence score is between ± 0.6 — hard rule enforced in prompt
- System prompt includes trading persona: veteran NSE intraday trader, risk-first mindset
- Temperature set to 0.1 for deterministic, consistent signal logic
- Response must be valid JSON only — use structured output parsing with fallback
- If Claude API fails, system falls back to rule-based signal (confluence score only) and flags signal as 'AI_UNAVAILABLE'

3.4 Module 4 — Paper Trading Engine

3.4.1 Order Simulation

- On BUY signal: simulate long entry at next candle open price (no look-ahead)
- On SELL signal: simulate short entry at next candle open price
- Track simulated fills: entry price, timestamp, instrument, quantity (₹1L notional per trade)
- Auto SL: place simulated stop at Claude-suggested SL level
- Auto Target: place simulated target orders at T1 and T2 levels
- Exit triggers: SL hit, T1/T2 hit, or EOD square-off at 03:10 PM

3.4.2 Position Management

- Maximum 2 open positions simultaneously (one per instrument)
- No pyramiding in Phase 1 — one signal per instrument at a time

- Trailing stop activation: when T1 hit, trail SL to entry (breakeven)
- Force-close all positions at 03:10 PM regardless of P&L

3.5 Module 5 — Analytics & Reporting Dashboard

Real-Time Dashboard Panels

- Live Signal Feed: rolling list of BUY/SELL/HOLD signals with timestamp, instrument, confidence
- Open Positions: current paper positions with live unrealized P&L
- Trade Log: full history of all trades with entry/exit/SL/targets
- P&L Chart: cumulative paper account equity curve
- Accuracy Metrics: win rate, average R:R, profit factor, max drawdown, Sharpe ratio
- Indicator Dashboard: live values for all indicators for both instruments across timeframes

Daily/Weekly Reports

- EOD summary email/log: trades taken, P&L, accuracy, notable signals missed
- Weekly accuracy report comparing signal confidence vs actual outcome
- Signal quality heatmap: time-of-day performance analysis

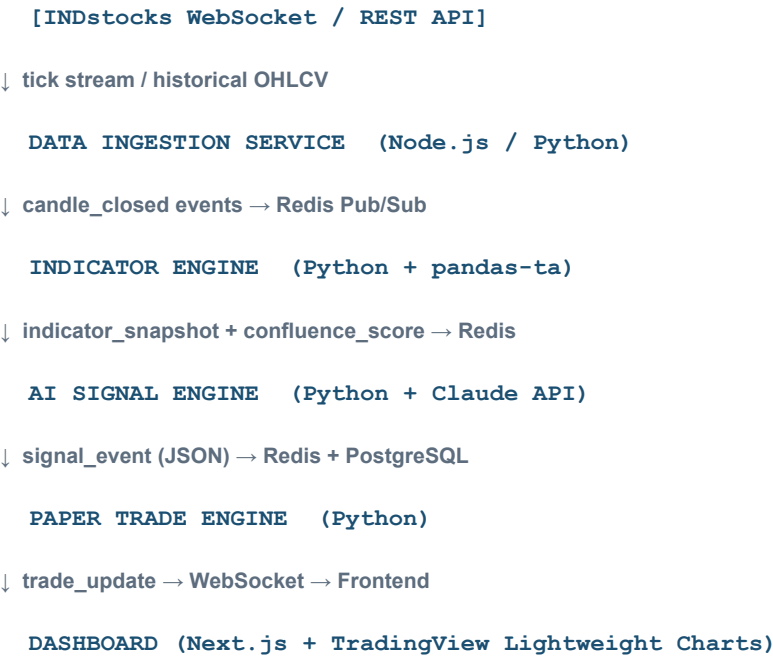
4. Non-Functional Requirements

Category	Requirement	Target SLA
Latency	Signal generation from candle close to output	< 5 seconds
Uptime	System availability during market hours	> 99.5%
Data Freshness	WebSocket tick lag from exchange	< 500ms
AI Response	Claude API call turnaround	< 4 seconds
DB Write	Candle + indicator write to DB	< 200ms
Scalability	Support adding 50+ instruments in future	Config-driven
Observability	All signal events logged with full context	100% coverage
Recoverability	Resume from last candle on crash/restart	< 60s recovery

5. System Architecture Design

ChartEdge AI follows an event-driven microservices architecture. Each module is a standalone service communicating via an internal message broker (Redis Pub/Sub or BullMQ). This allows individual services to be scaled, replaced, or upgraded independently.

5.1 High-Level Architecture Overview



5.2 Service Breakdown

Service	Language/Stack	Responsibility
data-ingestion	Python 3.11 + asyncio	WebSocket tick collection, candle building, REST historical fetch, TimescaleDB writes
indicator-engine	Python + pandas-ta	Subscribes to candle_closed; computes all indicators; writes to Redis cache; publishes indicator_ready event
ai-signal-engine	Python + httpx	Subscribes to indicator_ready; builds Claude prompt; calls Claude API; parses JSON response; publishes signal_event
paper-trade-engine	Python	Subscribes to signal_event; manages simulated positions, SL/TP tracking, EOD square-off; writes trades to PostgreSQL
analytics-service	Python + FastAPI	REST API for dashboard data: P&L, trade history, accuracy metrics, indicator values
notification-service	Python	EOD email/Slack alerts, daily summary report generation
api-gateway	Node.js + Express / FastAPI	Single entry point for frontend; auth, rate limit, route to downstream services
frontend-dashboard	Next.js 14 + TailwindCSS	Real-time signal dashboard, charts via TradingView Lightweight Charts, P&L panel, trade log

5.3 Data Flow — Intraday Signal Generation

- 1. 09:25 AM: data-ingestion connects WebSocket, subscribes to Nifty50 + BankNifty ticks
- 2. Each tick: accumulate into 1-min OHLCV candle in memory
- 3. On minute boundary: finalize candle, write to TimescaleDB, publish candle_closed:{instrument}:{timeframe} to Redis
- 4. indicator-engine receives candle_closed, loads last N candles from DB, computes all indicators, writes snapshot to Redis, publishes indicator_ready event with confluence score
- 5. ai-signal-engine: if |confluence_score| >= 0.6, build prompt payload from Redis snapshot, call Claude API
- 6. Claude returns JSON signal; ai-signal-engine validates schema, writes to PostgreSQL signals table, publishes signal_event to Redis
- 7. paper-trade-engine receives signal_event, simulates fill at next candle open, updates positions table
- 8. analytics-service detects new signal via DB trigger or Redis subscription; pushes update to frontend via WebSocket
- 9. Frontend dashboard updates signal feed, positions panel, P&L chart in real-time

5.4 Database Schema

TimescaleDB — candles (hypertable)

```
candles(time TIMESTAMPTZ, instrument TEXT, timeframe TEXT, open FLOAT, high FLOAT, low FLOAT, close FLOAT, volume BIGINT)
```

PostgreSQL — signals

```
signals(id UUID, created_at TIMESTAMPTZ, instrument TEXT, signal TEXT, confidence INT, entry_low FLOAT, entry_high FLOAT, stop_loss FLOAT, target_1 FLOAT, target_2 FLOAT, rrr FLOAT, reasoning TEXT, warnings JSONB, indicator_snapshot JSONB, confluence_score FLOAT, ai_model TEXT)
```

PostgreSQL — paper_trades

```
paper_trades(id UUID, signal_id UUID FK, instrument TEXT, direction TEXT, entry_price FLOAT, entry_time TIMESTAMPTZ, exit_price FLOAT, exit_time TIMESTAMPTZ, exit_reason TEXT, pnl FLOAT, status TEXT, sl_price FLOAT, t1_price FLOAT, t2_price FLOAT)
```

5.5 Technology Stack

Layer	Technology	Rationale
Data Ingestion	Python asyncio + websockets	Async tick handling, low latency candle building
Time-Series DB	TimescaleDB (PostgreSQL extension)	Native OHLCV queries, partitioning by time, SQL familiarity
Relational DB	PostgreSQL 15	Signals, trades, config — ACID compliant
Cache / Messaging	Redis 7 (Pub/Sub + Hash)	Sub-millisecond indicator cache, event bus between services
Indicator Math	Python pandas-ta / TA-Lib	Battle-tested, 130+ indicators, vectorized computation
AI Engine	Claude claude-sonnet via Anthropic SDK	Best-in-class reasoning, structured JSON output, low hallucination

Layer	Technology	Rationale
Task Queue	BullMQ (Redis-based)	Retry logic for Claude API calls, job prioritization
API Layer	FastAPI (Python)	Async REST, OpenAPI docs auto-generated, WebSocket support
Frontend	Next.js 14 + TailwindCSS	SSR + real-time updates via WebSocket, fast dashboard
Charts	TradingView Lightweight Charts	Professional candlestick charts with indicator overlays
Containerization	Docker + Docker Compose	Reproducible dev/prod parity, easy service orchestration
Monitoring	Prometheus + Grafana	Service health, latency tracking, Redis queue depth
Logging	Structured JSON logs → Loki	Searchable logs per signal_id, full audit trail

5.6 Configuration & Extensibility Design

All instrument definitions, indicator parameters, confluence weights, and AI model settings are stored in a single `config.yaml` file. Adding a new instrument or indicator requires only a config change and does not require code modification in any service.

```
instruments:
  - symbol: NIFTY
    exchange: NSE
    enabled: true
    timeframes: [1m, 5m, 15m, 1hr, 1D]
  - symbol: BANKNIFTY
    exchange: NSE
    enabled: true
    timeframes: [1m, 5m, 15m, 1hr, 1D]

confluence_thresholds:
  buy_threshold: 0.60
  sell_threshold: -0.60
```



```
indicator_weights:

  NIFTY:

    rsi: 0.20

    macd: 0.20

    ema_ribbon: 0.20

    vwap: 0.15

    supertrend: 0.15

    volume: 0.10


ai:

  model: claude-sonnet-4-20250514

  temperature: 0.1

  max_tokens: 500
```

6. Implementation Roadmap

Phase	Duration	Deliverables	Exit Criteria
Phase 0: Foundation	Week 1–2	INDstocks API integration, WebSocket candle builder, TimescaleDB setup, historical data loader	Both instruments streaming live candles to DB
Phase 1: Indicators	Week 2–3	Full indicator engine, Redis cache, confluence scoring, unit tests for all indicators	All 10 indicators computing correctly vs reference TradingView values
Phase 2: AI Signals	Week 3–4	Claude API integration, prompt builder, JSON parser, signal storage, fallback logic	Signal latency < 5s; AI signals matching manual analysis on 10 historical examples
Phase 3: Paper Trading	Week 4–5	Paper trade engine, SL/TP simulation, EOD square-off, P&L tracking, trade log	Full trade loop simulated for 5 days without errors
Phase 4: Dashboard	Week 5–7	Next.js dashboard, live signal feed, charts, P&L panel, accuracy metrics	All panels rendering live data; no data staleness > 10s
Phase 5: Live Paper Run	Month 2	30-day live paper trading session, daily P&L tracking, weekly accuracy review, tuning	Min 40% win rate; P&L breakeven or positive; system uptime > 99%

Phase	Duration	Deliverables	Exit Criteria
Phase 6: Live Trading	Month 3+	INDMoney API integration, risk limits, kill-switch, compliance review	Successful paper→live migration with < ₹500 slippage on first 10 trades

7. Risk Management & Compliance

7.1 System Risk Controls

- Kill switch: single API endpoint to halt all signal generation and close all paper positions immediately
- Max drawdown circuit breaker: if paper account drops > 5% in a single day, system pauses signals until next market day
- Signal rate limiter: maximum 3 signals per instrument per hour to prevent overtrading
- Confidence floor: signals below 60% confidence are logged but not executed in paper engine
- Duplicate signal prevention: if position already open for an instrument, new signals are queued not executed

7.2 Data Quality Controls

- Stale data detection: if no tick received for > 60s during market hours, flag as FEED_STALE and suppress signals
- Price sanity check: reject candles where OHLCV values deviate > 5% from last close (circuit filter proxy)
- Indicator validity check: NaN indicator values result in automatic HOLD, never a directional signal

7.3 Phase 2 Live Trading Risk (Future)

- Maximum position size per trade: ₹1,00,000 notional
- Maximum daily loss limit: ₹5,000 — hard stop, no override
- Manual review required before enabling live trading after paper phase
- Regulatory: Ensure compliance with SEBI algo trading guidelines before live deployment

8. INDstocks API Integration Reference

Endpoint / Feature	Usage in ChartEdge	Notes
GET /historical-data	Pre-load 90-day OHLCV on startup; backfill gaps	Fetch per instrument per timeframe; cache locally
GET /market-quote	Fallback real-time quote if WS disconnects	REST polling at 5s interval as backup only
WebSocket /ws/market	Primary real-time tick stream for both instruments	Subscribe on connect; handle reconnect gracefully

Endpoint / Feature	Usage in ChartEdge	Notes
GET /instruments	Validate instrument tokens for NIFTY and BANKNIFTY	Run once on startup; cache token mapping
POST /orders (Phase 2)	Live order placement via INDMoney API (future)	Not used in Phase 1 paper trading

9. AI Prompt Engineering Specification

9.1 System Prompt

The following system prompt should be used for all Claude signal generation calls:

You are a veteran NSE intraday technical trader with 15+ years experience trading Nifty 50 and Bank Nifty. You built consistent profits using pure technical chart analysis. You are risk-first: protecting capital is more important than catching every move. You analyse multi-timeframe confluence and never trade on a single indicator.

Rules you ALWAYS follow:

1. Never give a directional signal if confluence_score is between -0.60 and +0.60. Output HOLD.
2. Always size SL based on ATR, not arbitrary points.
3. Minimum R:R must be 1.5:1 to recommend entry.
4. Respond ONLY with valid JSON matching the exact schema provided. No preamble, no markdown.

9.2 User Prompt Template

Instrument: {instrument}

Current Time (IST): {timestamp}

Session Phase: {morning|midday|late}

=== 15-MIN CANDLE (CURRENT) ===

Open: {open} | High: {high} | Low: {low} | Close: {close} | Volume: {volume}

=== INDICATOR VALUES ===

```

RSI(14) 15m: {rsi_15m} [{rsi_state}]

MACD 15m: MACD={macd}, Signal={signal}, Hist={hist} [{macd_state}]

BB(20,2) 15m: Upper={bb_upper}, Mid={bb_mid}, Lower={bb_lower}, %B={pct_b}

EMA: 9={ema9}, 21={ema21}, 50={ema50}, 200={ema200} [{ema_state}]

VWAP: {vwap} | Price vs VWAP: {above|below}

ATR(14): {atr}

Supertrend: {supertrend_direction} @ {supertrend_level}

Stoch RSI: K={k}, D={d} [{stochrsi_state}]

Volume: {volume} vs 20MA: {vol_ma} [{volume_state}]

Daily Pivot: CPR={cpr}, R1={r1}, S1={s1}

=== CONFLUENCE SCORE ===

Score: {confluence_score} (range -1 to +1)

Individual votes: {indicator_votes_json}

=== HIGHER TIMEFRAME ===

1HR Trend: {trend_1hr}

Daily Trend: {trend_daily}

=== LAST 10 CANDLES (15m, OHLCV) ===

{candles_array_json}

Respond ONLY with this JSON schema:

{"signal":"BUY|SELL|HOLD","confidence":0-100,"entry_zone":
{"low":0,"high":0},"stop_loss":0,"target_1":0,"target_2":0,"risk_reward_ratio":0.0,"reasoning":"...","wa
rnings":[],"invalidation":"..."}

```

10. Recommended Project Folder Structure

```
chartedge-ai/
├── services/
│   ├── data-ingestion/           # WebSocket + REST + candle builder
│   │   ├── main.py
│   │   ├── ws_client.py
│   │   ├── candle_builder.py
│   │   └── historical_loader.py
│   ├── indicator-engine/        # All TA computations
│   │   ├── main.py
│   │   ├── indicators/
│   │   │   ├── rsi.py
│   │   │   ├── macd.py
│   │   │   ├── bollinger.py
│   │   │   ├── ema_ribbon.py
│   │   │   ├── vwap.py
│   │   │   ├── atr.py
│   │   │   ├── supertrend.py
│   │   │   └── pivot_points.py
│   │   └── confluence.py
│   ├── ai-signal-engine/       # Claude API integration
│   │   ├── main.py
│   │   ├── prompt_builder.py
│   │   └── claude_client.py
```

```
| | └─ signal_validator.py
| └─ paper-trade-engine/      # Simulated order book
| | └─ main.py
| | └─ position_manager.py
| | └─ pnl_tracker.py
| └─ analytics-service/      # REST API for dashboard
| | └─ main.py
| | └─ routers/
| └─ notification-service/    # Alerts + reports
└─ frontend/                 # Next.js dashboard
  └─ app/
    └─ components/
      └─ SignalFeed.tsx
      └─ PositionsPanel.tsx
      └─ PnLChart.tsx
      └─ IndicatorPanel.tsx
      └─ TradeLog.tsx
    └─ lib/
  └─ shared/
    └─ config.yaml           # Master config
    └─ db/
      └─ migrations/
      └─ schema.sql
```

	└─ models/
	└─ docker-compose.yml
	└─ docker-compose.dev.yml
	└─ README.md

Appendix — Glossary

Term	Definition
OHLCV	Open, High, Low, Close, Volume — standard candlestick data format
Confluence	Multiple indicators agreeing on the same directional bias, increasing signal reliability
ATR	Average True Range — measures volatility; used for dynamic SL/TP sizing
VWAP	Volume Weighted Average Price — intraday fair value benchmark
Supertrend	ATR-based trend-following indicator with clear directional flip signals
CPR	Central Pivot Range — key intraday S/R zone derived from previous day's HLC
R:R	Risk-to-Reward Ratio — $(\text{Target} - \text{Entry}) / (\text{Entry} - \text{Stop Loss})$
Paper Trading	Simulated trading with fake capital to test strategy without financial risk
EOD Square-off	Forced closure of all intraday positions before market close
GTT	Good Till Triggered — conditional orders that persist until price condition is met